DECK 7 OF 12 · FRC JAVA CRASH COURSE

# REV vs CTRE motors and controllers

What each one does on its own in 2026, and how to choose

WPILib 2026 · Java · Command-based

OVERVIEW

# What this deck covers

Deck 2 used a SPARK MAX. Deck 6 ran PID on the roboRIO. This deck moves that loop onto the motor controller.

- Which motors work with which controllers.
- Why a fast loop on the controller beats a slow one on the roboRIO.
- The same arm in REVLib and in Phoenix 6.
- What costs money, and how to choose.

HARDWARE

# The two vendors at a glance

| | REV | CTRE |
|---|---|---|
| Motors on sale | NEO 2.0, NEO 550, NEO Vortex | Kraken X60, Kraken X44, Minion |
| Controllers | SPARK MAX, SPARK Flex | Talon FX, Talon FXS |
| Built in or separate | Separate. SPARK Flex docks straight onto a Vortex | A Kraken has a Talon FX built in. Minion uses a separate Talon FXS |
| Encoders | Through Bore Encoder V2, MAXSpline Encoder (wired or CAN) | CANcoder (CAN) |
| Other sensors | Color Sensor V3, 2m Distance Sensor | Pigeon 2 (IMU), CANrange (distance), CANdi (inputs on CAN) |
| Setup app | REV Hardware Client 2 | Phoenix Tuner X |
| 2026 software | REVLib 2026.0.5, with 2026 firmware | Phoenix 6 26.3.0, with 2026 firmware |

Checked 2026-11-04. NEO V1.1 is discontinued and Falcon 500 is out of stock, but both are still legal if you own them.

HARDWARE

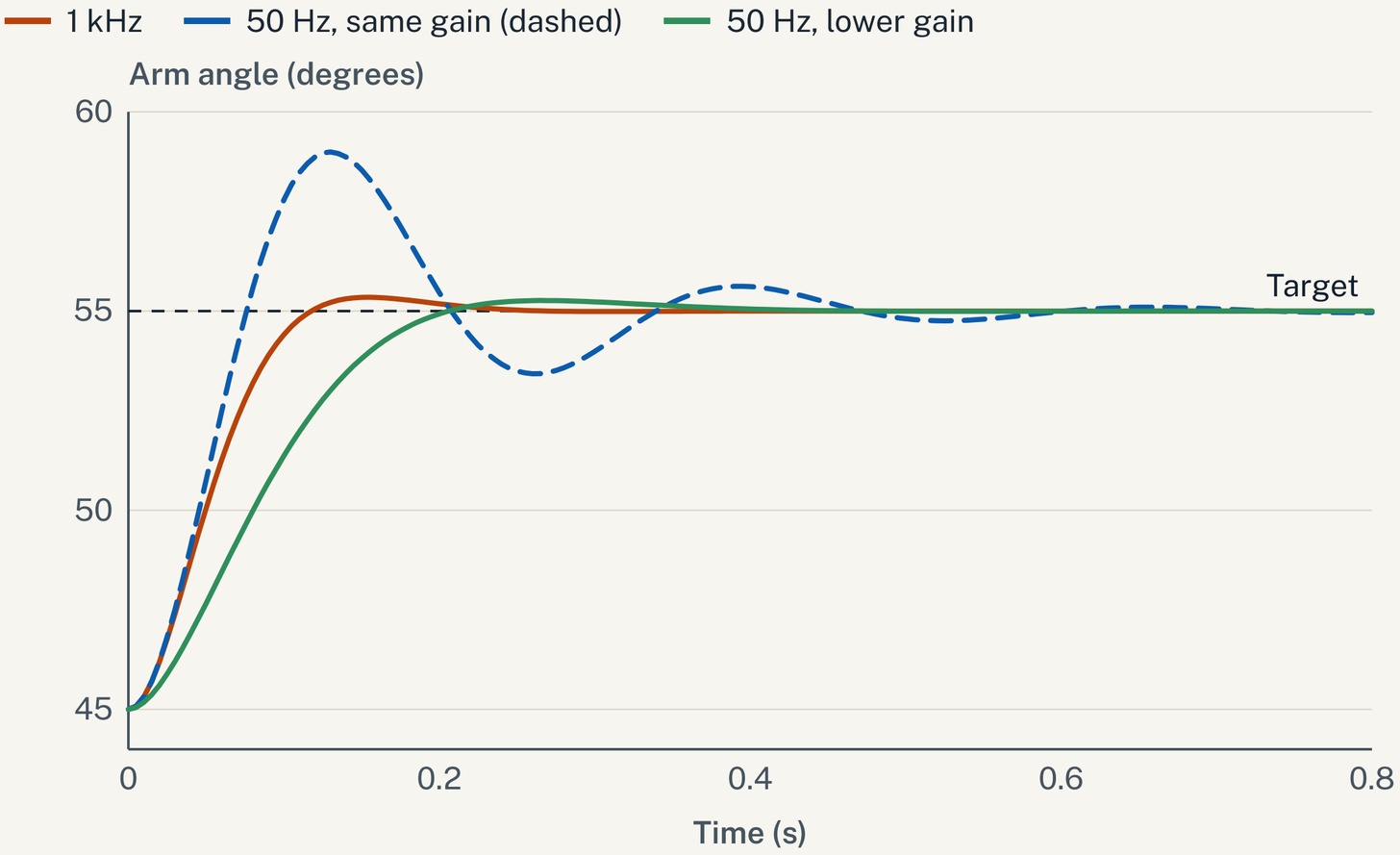
# Which motor works with which controller

| Motor | SPARK MAX | SPARK Flex | Talon FXS | Talon FX |
|---|---|---|---|---|
| NEO V1.1, NEO 550 | Yes | With Flex Dock | Yes | No |
| NEO 2.0 | Yes | With Flex Dock | Not listed by CTRE | No |
| NEO Vortex | With Solo Adapter | Docks directly | With Solo Adapter | No |
| Minion | Not documented | Not documented | Yes | No |
| Kraken X60, X44, Falcon | No | No | No | Built in |
| Brushed, such as CIM | Yes | With Flex Dock | Yes | No |

Tell the controller what is attached. Brushed Mode with a brushless motor connected permanently damages the motor. A Talon FXS will not drive until you pick its motor in MotorArrangement.

CONTROL THEORY

# Why a 1 kHz loop beats a 50 Hz loop




1. **Same gain at 50 Hz**
   The arm swings 4° past and takes 0.42 s to settle.

2. **Lower gain at 50 Hz**
   The swing goes away, but settling takes 0.17 s.

3. **Same gain at 1 kHz**
   No big swing, and it settles in 0.10 s.

ON THE CONTROLLER

# Closed loop on the controller

| | SPARK | Talon |
|---|---|---|
| Modes | Position, velocity, current, MAXMotion | Position, velocity, Motion Magic, in volts or duty cycle |
| Loop rate | Every 1 ms | 1 kHz, so also every 1 ms |
| Feedforward | kS, kV, kA, plus an arbitrary term from your code | kS, kV, kA, plus a FeedForward term in the request |
| Gravity | kG for elevators. kCos with kCosRatio for arms | kG with GravityType. GravityArmPositionOffset shifts the arm angle |
| Gain slots | 4, chosen in setSetpoint | 3, plus simple gain scheduling on error |
| Arrived? | isAtSetpoint() | ClosedLoopError, and MotionMagicAtTarget for the profile |
| Following | config.follow(leader, invert), set on the follower | Follower request: copies the leader’s output type |

Arm gravity works the same way on both: the sensor must read 0 when the arm is level.

ON THE CONTROLLER

# Motion profiles on the controller

## MAXMotion (REV)

**Position:** a trapezoid. Speed up, cruise, slow down.

You set cruise velocity, max acceleration and allowed profile error.

Drift too far off the plan and the SPARK makes a new plan from where it is.

**Velocity:** ramps to a speed at your max acceleration.

## Motion Magic (CTRE)

**Motion Magic:** a trapezoid, or a smoother S-curve if you set jerk.

**Velocity:** ramps to a speed at your acceleration, with optional jerk.

**Expo:** an exponential curve from kV and kA that fits what the motor can do.

**Dynamic:** change the limits in every request. Needs Pro and a CANivore.

ON THE CONTROLLER

# Limits and protection

| | SPARK | Talon |
|---|---|---|
| Motor current | Smart Current Limit. Holds phase current. Default 80 A | Stator limit. Default 120 A, on |
| Battery current | No separate setting | Supply limit. Default 70 A, then 40 A after 1 s, on |
| Soft limits | Forward and reverse, in softLimit | Forward and reverse, in SoftwareLimitSwitch. Position targets get capped to them |
| Limit switches | Wired to the data port | Talon FXS: wired. Kraken X60: none, so use a CANcoder, CANdi or CANrange |
| Battery sag | voltageCompensation holds output steady | Voltage requests do this, up to what the battery gives |

Brownout is the same on both. Below 6.3 V (roboRIO 1) or 6.75 V (roboRIO 2 default), the roboRIO sends CAN motor controllers an explicit disable.

BEST PRACTICE

# Run the loop on the controller, limit every motor

> **If timing matters, run the loop on the controller. On every motor, set a current limit.**

The controller reads a wired sensor directly, every millisecond. And the 80 A SPARK default is twice the top of REV's suggested range for a NEO 550, which is 20 A to 40 A.

```
// SPARK
config.smartCurrentLimit(40);
// Talon FX
config.CurrentLimits.StatorCurrentLimit = 40;
config.CurrentLimits.StatorCurrentLimitEnable = true;
```

COST

# Free features and paid features

**REV: one tier**
REVLib, Hardware Client 2, StatusLogger, MAXMotion and simulation. REV's docs list no paid software.

**Phoenix 6: free**
Volt and duty cycle closed loops. Motion Magic, Velocity and Expo. Swerve API. Tuner X. Simulation. Limited log export.

**CANivore: hardware**
A second CAN bus for CTRE devices, with CAN FD and CANivore timestamps.

**Pro license**
FOC and torque control on Talon FX. FOC gives about 15% more peak power. FusedCANcoder, SyncCANcoder, full log export, hoot replay.

**Pro and CANivore**
Timesync, device timestamps, Dynamic Motion Magic, full differential control.

Without Pro, a request with EnableFOC falls back to normal drive, but TorqueCurrentFOC will not run. A Season Pass covers 100 devices for one team number. Pro features are free in simulation.

TOOLS

# Logging, tools and simulation

| | REV | CTRE |
|---|---|---|
| Automatic log | StatusLogger: on by default, writes .revlog | Signal Logger: writes .hoot. Auto-starts on roboRIO 2, or roboRIO 1 with a USB drive |
| Read it | Opens directly in AdvantageScope | AdvantageScope or Tuner X. Export without Pro is a limited signal list |
| SysId | WPILib SysId | WPILib SysId, fed by Signal Logger |
| Setup app | Hardware Client 2: firmware, CAN IDs, run and plot motors, zero encoders | Tuner X: firmware, configs, plots, self test, log tools |
| Generates | No code generator. Templates for MAXSwerve and EasySwerve | Swerve project, Elevator subsystem, CANdle subsystem |
| Simulation | SparkMaxSim and SparkFlexSim run closed loop and MAXMotion | SimState classes for every Phoenix 6 device |

SAME JOB, BOTH VENDORS

# Arm setup: sensor, gains and gravity

**REVLib 2026**

```
config.absoluteEncoder
    .apply(AbsoluteEncoderConfig.Presets
        .REV_ThroughBoreEncoderV2)
    .zeroOffset(ENCODER_OFFSET_ROT);

config.closedLoop
    .feedbackSensor(
        FeedbackSensor.kAbsoluteEncoder)
    .p(KP)
    .feedForward
    .kV(KV)
    .kCos(KG).kCosRatio(1.0);
```

**Phoenix 6 2026**

```
var feedback = config.Feedback;
feedback.FeedbackRemoteSensorID = ENCODER_ID;
feedback.FeedbackSensorSource =
    FeedbackSensorSourceValue.RemoteCANcoder;
feedback.SensorToMechanismRatio = 1.0;
feedback.RotorToSensorRatio = GEAR_RATIO;

config.Slot0.GravityType =
    GravityTypeValue.Arm_Cosine;
config.Slot0.kG = KG;
config.Slot0.kP = KP;
config.Slot0.kV = KV;
```

SAME JOB, BOTH VENDORS

# Arm move: profile and command

**REVLib 2026**

```
config.closedLoop.maxMotion
    .cruiseVelocity(CRUISE_RPM)
    .maxAcceleration(ACCEL_RPM_PER_S)
    .allowedProfileError(PROFILE_ERROR_ROT);

public Command moveTo(double degrees) {
  double rotations = degrees / 360.0;
  return runOnce(() -> {
    target = rotations;
    controller.setSetpoint(rotations,
      ControlType.kMAXMotionPositionControl);
  }).andThen(waitUntil(this::atTarget));
}
```

**Phoenix 6 2026**

```
var magic = config.MotionMagic;
magic.MotionMagicCruiseVelocity = CRUISE_RPS;
magic.MotionMagicAcceleration = ACCEL_RPS2;

public Command moveTo(double degrees) {
  double rotations = degrees / 360.0;
  return runOnce(() -> {
    target = rotations;
    motor.setControl(
        request.withPosition(rotations));
  }).andThen(waitUntil(this::atTarget));
}
```

COMMON MISTAKE

# Newest firmware is not always the right firmware

| | |
|---|---|
| **What you see** | A SPARK shows “Outdated REV Device”, REVLib says the firmware is too old or too new, or Phoenix reports a firmware version error. |
| **Why it happens** | On both vendors, firmware and library are a matched pair for each season. The newest firmware listed can be for next season. |
| **The fix** | For 2026, pick the newest 26.x in Hardware Client 2, and the 2026 firmware year in Tuner X. |

DECISION

# How a young team can choose

### Budget

Count whole systems. A Kraken includes its controller. A NEO needs a SPARK, and a Minion needs a Talon FXS.

### What you own

Start from your shelf. KitBot SPARK MAX controllers, NEO V1.1 and Falcon 500 are all still legal.

### Who can help

Choose what your team, or a team nearby, can debug at an event.

### Mixing is allowed

R504 lists both vendors and one project can use both libraries. You pay with two apps and two update routines.

EXERCISE

# Pick the parts for one mechanism

1. Choose one mechanism: an arm, a shooter or an intake.
2. Pick a motor and a controller. Check them on the pairing table.
3. Write down its current limit and where its sensor plugs in.
4. Open RevArm.java or CtreArm.java and find the gravity lines.

**Done when: you have a motor, a controller, a current limit and a sensor, and have found the gravity lines.**

NEXT SEASON

# What changes for 2027

| In 2026 | Changing for 2027 |
|---|---|
| SPARK firmware 26.x, conversion factors | Shipped already: firmware 27.0.0, conversion factors gone |
| Phoenix firmware for 2026 | Alpha: Phoenix 26.70 needs 26.70.x firmware, no roboRIO |
| new TalonFX(id, bus) on a roboRIO | Alpha: int and String constructors go. CANBus takes a CANPort |
| new SparkMax(id, type) | Alpha: constructors take a WPILib CANPort |
| REV getters return plain values | Alpha: getters return a Signal, so call .get() |
| motor.set() on a Talon | Alpha: renamed setThrottle() |

Firmware 27.0.0 is a normal REV release from 2026-10-29. Alpha items may change: REVLib 2027.0.0-alpha-7, Phoenix 6 26.70.0-alpha-2.

RECAP

# What this deck covered

- Which motors work with which controllers.
- Why a 1 kHz loop on the controller beats a 50 Hz loop on the roboRIO.
- The same arm in REVLib and in Phoenix 6.
- What needs Phoenix Pro or a CANivore, and what does not.
- Choosing from budget, parts and available help.

**Next: That ends the core course. Decks 8 to 10 go deeper.**

REFERENCE

# Official docs

These change every season, so check them before you buy.

**SPARK closed loop**

docs.revrobotics.com

**REV Hardware Client 2**

docs.revrobotics.com

**Phoenix 6 free and Pro**

v6.docs.ctr-electronics.com

**Talon FXS motors**

v6.docs.ctr-electronics.com